

FDE Candidate Technical Exercise

The Exercise

You are joining an existing engineering project and have been asked to ship a small product improvement.

Your job is to understand the codebase, implement the change, test it, and leave the repository in better shape than you found it.

We expect you to use modern coding agents as part of your workflow. Which tools you use—and how you use them—is up to you.

Suggested time: ~90 minutes

Repository: <https://github.com/Rudge/kotlin-ktor-realworld-example-app>

Fork the repository to your personal GitHub account. Your fork is your submission.

The Codebase

This is a RealWorld implementation built with Kotlin, Ktor, Exposed, and H2. It implements a Medium-like blogging API with authentication, articles, comments, profiles, favorites, and pagination.

You should be able to get started with:

```
./gradlew clean build  
./gradlew test  
./gradlew run
```

Spend enough time understanding the existing system before deciding how to make your changes.

1. Ship a Feature

Choose one of the following.

Popular Articles

Add:

GET /api/articles/feed/popular

Return articles ordered by number of favorites, most popular first.

Support limit and offset pagination.

User Activity

Add:

GET /api/profiles/:username/stats

Return:

```
{
  "stats": {
    "articlesCount": 5,
    "commentsCount": 12,
    "favoritesCount": 3
  }
}
```

Article Search

Add:

GET /api/articles/search?q=<term>

Search article titles and body content and return results using the existing article-list response format.

You may propose a different feature of similar scope if you think it would make for a better exercise.

Whatever you choose should feel native to the existing application. Follow existing conventions where they make sense and preserve existing behavior.

2. Prove It Works

Add the tests you believe are necessary to confidently ship your change.

We should be able to understand from the repository:

- What behavior you intended
- What important edge cases you considered
- Whether the implementation works
- Whether existing behavior still works

How you establish that confidence is part of the exercise.

3. Work Agentically

Use one or more coding-agent harnesses while completing the exercise.

We are intentionally not prescribing a particular tool, model, prompting strategy, or workflow.

Treat the agents as engineering resources you can direct as you see fit.

We will be interested in how you used them, what you trusted them with, and how you decided their work was correct.

4. Make CI Useful

The repository should have a GitHub Actions pipeline appropriate for the change you are shipping.

At minimum, CI should:

- Build and test the project
- Run against JDK 17 and JDK 21
- Surface test failures clearly
- Make sensible use of Gradle caching

Bonus: Run the included RealWorld API specification tests against the application.

Beyond that, use your judgment.

5. Leave Us Your Work Log

Add an AGENT_WORKLOG.md file to the repository.

Keep it short. We do not want a transcript.

Give us enough to understand:

- What agent harnesses/models you used
- A few representative instructions you gave them
- Where agents materially changed how you approached the problem
- How you verified their work
- Something they got wrong, missed, or caused you to reconsider
- What you would do differently next time

We're particularly interested in the gap between asking an agent to do something and building a workflow where you can trust the result.

6. Walk Us Through It

Record a 5–10 minute walkthrough of your submission.

Screen recording with voice-over is preferred; audio with enough context to follow your work is also fine.

Show us what you built and how you approached the problem. Spend some time on how agents fit into your workflow and anything interesting that happened along the way.

Add the recording link to AGENT_WORKLOG.md.

Production quality doesn't matter.

Submission

Send us the full URL of your forked repository.

Your fork should contain everything needed to evaluate the exercise, including:

- Your implementation
- Tests
- CI configuration

- AGENT_WORKLOG.md
- A link to your video or voice-over walkthrough

Please make sure we can access the repository and recording.

What We Care About

We are evaluating the overall engineering approach, not whether you checked the largest number of boxes.

Engineering

Does the solution fit the existing system? Is it understandable, appropriately scoped, and something you would be comfortable shipping?

Verification

How did you establish that the change works? Did you identify the right things to test and verify?

Agentic Engineering

Did agents materially improve your ability to understand, build, test, or review the system?

More agent usage is not necessarily better. We care about how effectively you use the capabilities available to you.

Judgment

Where did you investigate further? What did you delegate? What did you verify yourself? What trade-offs did you make under the time constraint?

Communication

Can another engineer understand what you changed, why you changed it, and how confident you are in it?

FDE Mindset

Could you take the workflow you used here into a customer environment—where the codebase is unfamiliar, requirements are imperfect, and getting to a reliable result matters more than producing code quickly?

A Few Notes

The H2 database is in-memory, so no external database should be required.

There is existing integration-test infrastructure in the repository. Use it—or don't—based on what you discover and what you think is appropriate.

The project currently targets JVM 16. Keep compatibility in mind when setting up the requested JDK 17/21 CI matrix.

If you encounter ambiguity, make a reasonable decision and document it rather than waiting for perfect requirements.

We don't expect everything to be perfect in 90 minutes. Prioritize, ship, and be prepared to explain your decisions.

Good luck.